

MINI ASSESSMENT

Reliable File Uploads and Handling Invalid API Data

Name: G. Bala Sri **Student ID:** 99230041245

Course: Gen-AI

Problem Statement

Build a robust Python client that uploads a file to an API and processes the response safely. The client must handle temporary server failures, unexpected response formats, and missing fields without crashing. It should retry temporary failures with a bounded delay, validate the response before using it, and provide safe defaults when optional data is missing.

Q1. If an API returns a temporary server error such as HTTP 503, what should your program do?

HTTP 503 means the service is temporarily unavailable. The client should not retry continuously. It should retry only a limited number of times, use a short delay that increases between attempts (exponential backoff), and respect the `Retry-After` header when the server provides one. If all retries fail, it should return a clear error or let the calling code handle the failure.

Example:

```
import time
import requests

def get_with_retry(url, retries=4):
    delay = 1
    for attempt in range(retries):
        response = requests.get(url, timeout=10)
        if response.status_code != 503:
            return response
        pause = int(response.headers.get('Retry-After', delay))
        time.sleep(pause)
        delay *= 2
    raise RuntimeError('Service unavailable after retries')
```

Why this matters: Repeated requests without a limit can overload the service. A bounded retry policy gives the server time to recover and prevents an endless loop.

Q2. What should your program do if the API response is not valid JSON?

A response may contain HTML, plain text, or an empty body instead of JSON. The program should check the HTTP status and content type when appropriate, then place JSON parsing inside a `try/except` block. If parsing fails, it should log useful details and use a safe fallback or return a controlled error rather than crashing.

Example:

```
response = requests.get(url, timeout=10)
try:
    payload = response.json()
except ValueError as error:
    print(f'Invalid JSON (HTTP {response.status_code}): {error}')
    print('Response preview:', response.text[:200])
    payload = {}
```

Why this matters: A controlled fallback keeps one malformed response from stopping the entire program and helps developers investigate the issue.

Q3. Why is dictionary `.get()` safer than square-bracket lookup for optional API fields?

Using `payload['status']` raises a `KeyError` if the key is missing. In contrast, `payload.get('status')` returns `None` by default, or you can provide a fallback such as `payload.get('status', 'unknown')`. This is useful when API fields are optional or may change.

Example:

```
payload = {'id': 205, 'filename': 'report.csv'}
status = payload.get('status', 'unknown')
```

```
print(status) # unknown
```

Why this matters: A missing optional field should be handled deliberately instead of causing an unexpected crash. Use square brackets when the field is required and its absence should be treated as an error.

Expected Outcome

The completed client should retry temporary service failures within a fixed limit, handle invalid JSON safely, and continue processing records when optional fields are missing. These practices make API integrations more reliable and easier to debug.